Teach Module 33: React Component Architecture

Source note: The bootcamp document was used only to identify Module 33's topic. The teaching content below is original instructional material.

Module Topic

Module 33 covers **React Component Architecture**.

The main idea is that a React user interface should be broken into small, reusable pieces called **components**.

Instead of building one giant page, you build pieces such as:

```
Header
Sidebar
ProductCard
SearchBar
UserProfile
Footer
```

Each component should own one clear part of the interface.

1. What Is A Component?

A React component is usually a JavaScript function that returns UI.

```
function WelcomeMessage() {
  return <h1>Welcome to React</h1>;
}
```

The HTML-like syntax is called **JSX**.

JSX lets you write UI-like syntax inside JavaScript.

```
const name = "Asha";

function Greeting() {
  return <p>Hello, {name}</p>;
}
```

The `{}` syntax lets you insert JavaScript expressions into the UI.

2. Functional Components

Modern React mostly uses **functional components**.

```
function UserCard() {
  return (
    <div>
      <h2>Maria Chen</h2>
```

```
      <p>Software Engineer</p>
    </div>
  );
}
```

A good component should usually be:

- Small
- Focused
- Reusable
- Easy to understand
- Named clearly

Less useful name:

```
function Box() {}
```

Better name:

```
function UserProfileCard() {}
```

3. Component Composition

Composition means building larger UIs by combining smaller components.

```
function App() {
  return (
    <main>
      <Header />
      <UserCard />
      <Footer />
    </main>
  );
}
```

Think of React as building with blocks. `App` is the whole screen, and components are the blocks.

A component can contain other components:

```
function Dashboard() {
  return (
    <>
      <StatsPanel />
      <RecentActivity />
      <Notifications />
    </>
  );
}
```

4. Props: Passing Data Into Components

Components become reusable when they receive data through **props**.

```
function UserCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.role}</p>
    </div>
  );
}
```

Use it like this:

```
<UserCard name="Maria" role="Backend Developer" />
<UserCard name="Dev" role="Frontend Developer" />
```

Same component, different data.

Most developers write props using destructuring:

```
function UserCard({ name, role }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}
```

5. UI Layering

A clean React app often has layers:

```
App
Page
Section
Component
Small UI element
```

Example:

```
App
  DashboardPage
    UserSummarySection
      UserCard
```

```
        Avatar
        Badge
```

This keeps the app organized.

A common mistake is making one huge component:

```
function Dashboard() {
  // header, sidebar, cards, tables, forms, buttons...
}
```

Better:

```
function Dashboard() {
  return (
    <>
      <DashboardHeader />
      <DashboardSidebar />
      <DashboardContent />
    </>
  );
}
```

6. Reusability Pattern

If you notice repeated UI, turn it into a component.

Repeated code:

```
<button>Save</button>
<button>Cancel</button>
<button>Delete</button>
```

Reusable component:

```
function Button({ label }) {
  return <button>{label}</button>;
}
```

Usage:

```
<Button label="Save" />
<Button label="Cancel" />
<Button label="Delete" />
```

Later, you can improve it:

```
function Button({ label, type }) {
  return <button className={type}>{label}</button>;
```

```
}
```

Mini Practice

Build this mentally first:

```
App
  Navbar
  CourseCard
  CourseCard
  CourseCard
```

The `CourseCard` should accept:

- `title`
- `level`
- `duration`

Example solution:

```jsx
function CourseCard({ title, level, duration }) {
  return (
    <article>
      <h2>{title}</h2>
      <p>Level: {level}</p>
      <p>Duration: {duration}</p>
    </article>
  );
}

function App() {
  return (
    <main>
      <Navbar />
      <CourseCard title="React Basics" level="Beginner" duration="3 hours" />
      <CourseCard title="Spring Boot APIs" level="Intermediate" duration="5 hours" />
      <CourseCard title="Docker Fundamentals" level="Beginner" duration="2 hours" />
    </main>
  );
}
```

Key takeaway: **React architecture is mostly about deciding what should become a component, how components fit together, and what data each component needs.**

Practice Exercises

Exercise 1: Profile Card

Create a `ProfileCard` component that receives:

```
name
role
location
avatarUrl
```

Render a simple card showing the user's avatar, name, role, and location.

Practice focus: functional components, props, JSX.

Exercise 2: Course Catalog

Create these components:

```
App
  CourseList
    CourseCard
```

Each `CourseCard` should display:

```
title
level
duration
instructor
```

Use an array of course objects and render multiple cards with `.map()`.

Practice focus: reusable components, rendering lists, composition.

Exercise 3: Product Card

Create a `ProductCard` component with:

```
name
price
category
inStock
```

If `inStock` is true, show `"Available"`. Otherwise show `"Out of stock"`.

Practice focus: conditional rendering.

Exercise 4: Dashboard Layout

Build this component structure:

```
Dashboard
  Header
  Sidebar
  MainContent
```

```
    StatsPanel
    ActivityList
```

You do not need real functionality. Focus on splitting the page into sensible components.

Practice focus: UI layering and component hierarchy.

Exercise 5: Reusable Button

Create a `Button` component that accepts:

```
label
variant
```

Example usage:

```
<Button label="Save" variant="primary" />
<Button label="Cancel" variant="secondary" />
<Button label="Delete" variant="danger" />
```

Use different CSS classes based on the `variant`.

Practice focus: reusable UI patterns.

Exercise 6: Blog Page

Create:

```
BlogPage
  BlogHeader
  BlogPostList
    BlogPostCard
```

Each blog post should have:

```
title
author
date
excerpt
```

Render at least three posts.

Practice focus: data-driven component composition.

Exercise 7: Component Refactor

Start with one large component called `UserDashboard` that contains everything: heading, stats, profile, notifications, and actions.

Then refactor it into:

```
UserDashboard
  DashboardHeader
  UserStats
  UserProfile
  NotificationsList
  ActionPanel
```

Practice focus: identifying component boundaries.

Exercise 8: Card Component With Children

Create a reusable `Card` component that uses `children`.

```
function Card({ children }) {
  return <div className="card">{children}</div>;
}
```

Use it like:

```
<Card>
  <h2>Account Summary</h2>
  <p>Your balance is $2,400.</p>
</Card>
```

Practice focus: component composition using `children`.

Best Capstone-Style Practice

Build a small **Learning Dashboard** with this structure:

```
App
  Navbar
  DashboardPage
    WelcomeBanner
    CourseProgressList
      CourseProgressCard
    UpcomingTasks
      TaskItem
    InstructorPanel
      InstructorCard
```

Use mock data arrays for courses, tasks, and instructors.

This gives practice with:

- JSX
- Functional components
- Props
- Reusable cards
- Parent-child structure

- UI layering
- Rendering repeated components

A good target is **8 to 12 components total**.

Module 33 Lab: React Component Architecture

Goal

Build a small React app called **Learning Dashboard** using clean component architecture.

You will practice:

- JSX
- Functional components
- Props
- Component composition
- Reusable UI components
- Parent-child structure
- Rendering lists

Lab Scenario

You are building a dashboard for a bootcamp student. The dashboard should show:

- A navigation bar
- A welcome section
- A list of enrolled courses
- Upcoming tasks
- Instructor information

Required Component Structure

```
App
  Navbar
  DashboardPage
    WelcomeBanner
    CourseProgressList
      CourseProgressCard
    UpcomingTasks
      TaskItem
    InstructorPanel
      InstructorCard
```

Step 1: Create The React App

If using Vite:

```
npm create vite@latest module-33-lab -- --template react
cd module-33-lab
npm install
npm run dev
```

Step 2: Create Mock Data

In `App.jsx`, create arrays like this:

```
const courses = [
  {
    id: 1,
    title: "React Component Architecture",
    progress: 70,
    level: "Beginner"
  },
  {
    id: 2,
    title: "State and Event Management",
    progress: 35,
    level: "Intermediate"
  },
  {
    id: 3,
    title: "Spring Boot APIs",
    progress: 90,
    level: "Intermediate"
  }
];

const tasks = [
  {
    id: 1,
    title: "Complete React components lab",
    dueDate: "Today"
  },
  {
    id: 2,
    title: "Review JSX syntax",
    dueDate: "Tomorrow"
  },
  {
    id: 3,
    title: "Build reusable Button component",
    dueDate: "Friday"
  }
];

const instructors = [
  {
    id: 1,
    name: "Avery Johnson",
    specialty: "Frontend Engineering"
  },
  {
    id: 2,
    name: "Maya Patel",
```

```
    specialty: "Java Backend Development"
  }
];
```

Step 3: Build Components

Navbar:

```
function Navbar() {
  return (
    <nav>
      <h1>Learning Dashboard</h1>
    </nav>
  );
}
```

Course card:

```
function CourseProgressCard({ title, progress, level }) {
  return (
    <article>
      <h3>{title}</h3>
      <p>Level: {level}</p>
      <p>Progress: {progress}%</p>
    </article>
  );
}
```

Course list:

```
function CourseProgressList({ courses }) {
  return (
    <section>
      <h2>My Courses</h2>

      {courses.map((course) => (
        <CourseProgressCard
          key={course.id}
          title={course.title}
          progress={course.progress}
          level={course.level}
        />
      ))}
    </section>
  );
}
```

Task item:

```
function TaskItem({ title, dueDate }) {
  return (
    <li>
      <strong>{title}</strong> - {dueDate}
    </li>
  );
}
```

Upcoming tasks:

```
function UpcomingTasks({ tasks }) {
  return (
    <section>
      <h2>Upcoming Tasks</h2>

      <ul>
        {tasks.map((task) => (
          <TaskItem
            key={task.id}
            title={task.title}
            dueDate={task.dueDate}
          />
        ))}
      </ul>
    </section>
  );
}
```

Instructor card:

```
function InstructorCard({ name, specialty }) {
  return (
    <article>
      <h3>{name}</h3>
      <p>{specialty}</p>
    </article>
  );
}
```

Instructor panel:

```
function InstructorPanel({ instructors }) {
  return (
    <section>
      <h2>Instructors</h2>

      {instructors.map((instructor) => (
        <InstructorCard
          key={instructor.id}
```

```
          name={instructor.name}
          specialty={instructor.specialty}
        />
      ))}
    </section>
  );
}
```

Dashboard page:

```
function WelcomeBanner({ studentName }) {
  return (
    <section>
      <h2>Welcome back, {studentName}</h2>
      <p>Here is your learning progress for today.</p>
    </section>
  );
}

function DashboardPage({ courses, tasks, instructors }) {
  return (
    <main>
      <WelcomeBanner studentName="Jordan" />
      <CourseProgressList courses={courses} />
      <UpcomingTasks tasks={tasks} />
      <InstructorPanel instructors={instructors} />
    </main>
  );
}
```

Final `App` :

```
function App() {
  return (
    <>
      <Navbar />
      <DashboardPage
        courses={courses}
        tasks={tasks}
        instructors={instructors}
      />
    </>
  );
}

export default App;
```

Step 4: Add Basic Styling

In `App.css` , add simple styles:

```css
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f6f8;
  color: #222;
}

nav {
  background: #1f2937;
  color: white;
  padding: 16px 24px;
}

main {
  max-width: 900px;
  margin: 24px auto;
  padding: 0 16px;
}

section {
  margin-bottom: 24px;
}

article {
  background: white;
  padding: 16px;
  margin-bottom: 12px;
  border-radius: 8px;
  border: 1px solid #ddd;
}
```

Lab Deliverables

By the end, you should have:

- At least 8 React components
- Props passed from parent to child
- At least two `.map()` list renderings
- A reusable card-style component pattern
- A clean component hierarchy

Challenge Extension

Add a reusable `Badge` component:

```jsx
function Badge({ label }) {
  return <span className="badge">{label}</span>;
}
```

Use it inside `CourseProgressCard` to display the course level.